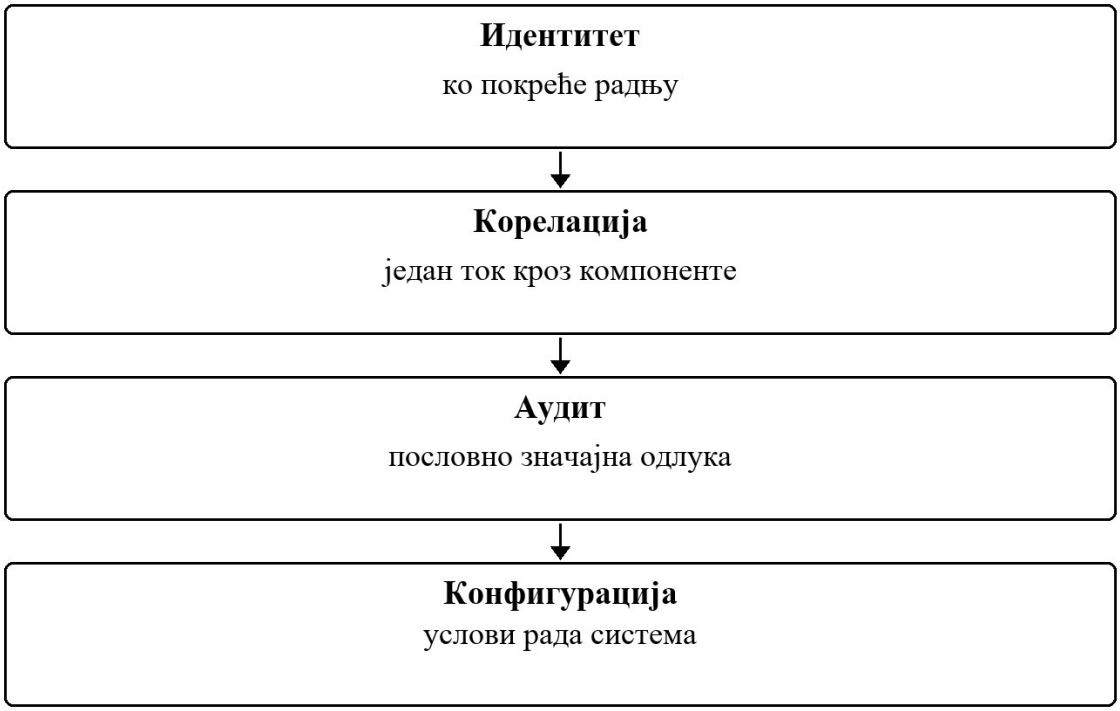
Идентитет, аудит, могућност посматрања система и конфигурација

Основе дистрибуираног програмирања · 03

Дистрибуирани систем мора моћи да одговори на четири питања: ко је покренуо радњу, шта се догодило, кроз које компоненте је ток прошао и под којим условима је радио. Идентитет, аудит, могућност посматрања система и конфигурација дају одговоре који се не смеју измишљати изнова у свакој функционалности.

Када се ток поквари, појединачни логови нису довољни. Заједнички корелациони идентификатор и јасан аудит траг омогућавају да тим састави причу о операцији без нагађања и без излагања осетљивих података.



Слика 1. Попречне могућности које свака компонента користи на исти начин

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

3.1 Видљивост мора бити уграђена

- Аудит чува одлуке које су важне за одговорност и истрагу.
- Логови и трагови чувају технички контекст, али не тајне.
- Конфигурација има валидно значење, власника и начин промене.

Питање за дискусију

Како бисте реконструисали једну неуспелу операцију која је прошла кроз Програмски интерфејс, ред порука и воркер, а да не читате неограничену количину логова?

3.2 Како се ова тема види у развоју

Тим бира неколико операција које мора умети да прати од захтева до исхода. Свака компонента преноси заједнички контекст, а тестови проверавају да промена конфигурације или прекид тока оставља довољно информација за разумевање.

Честе грешке које треба препознати

- Лог који открива креденцијале или личне податке.
- Корелациони идентификатор креиран у свакој компоненти уместо пренет.
- Конфигурација прихваћена без провере опсега и утицаја.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

3.3 Пример: захтев пролази кроз три компоненте

Корисник покреће операцију кроз Програмски интерфејс, Програмски интерфејс објављује поруку, а воркер завршава обраду. Ако свака компонента бележи свој неповезан идентификатор, тим не може брзо да одговори да ли је захтев прихваћен, обрађен или изгубљен између корака. Заједнички контекст претвара три засебна записа у једну проверљиву причу.

- Корелациони идентификатор настаје на граници тока и преноси се даље.
- Аудит чува одлуке које утичу на одговорност, док лог чува технички контекст.
- Конфигурација одређује очекивано понашање, као што су рокови и прагови, и мора бити проверљива.

Како се сценарио анализира

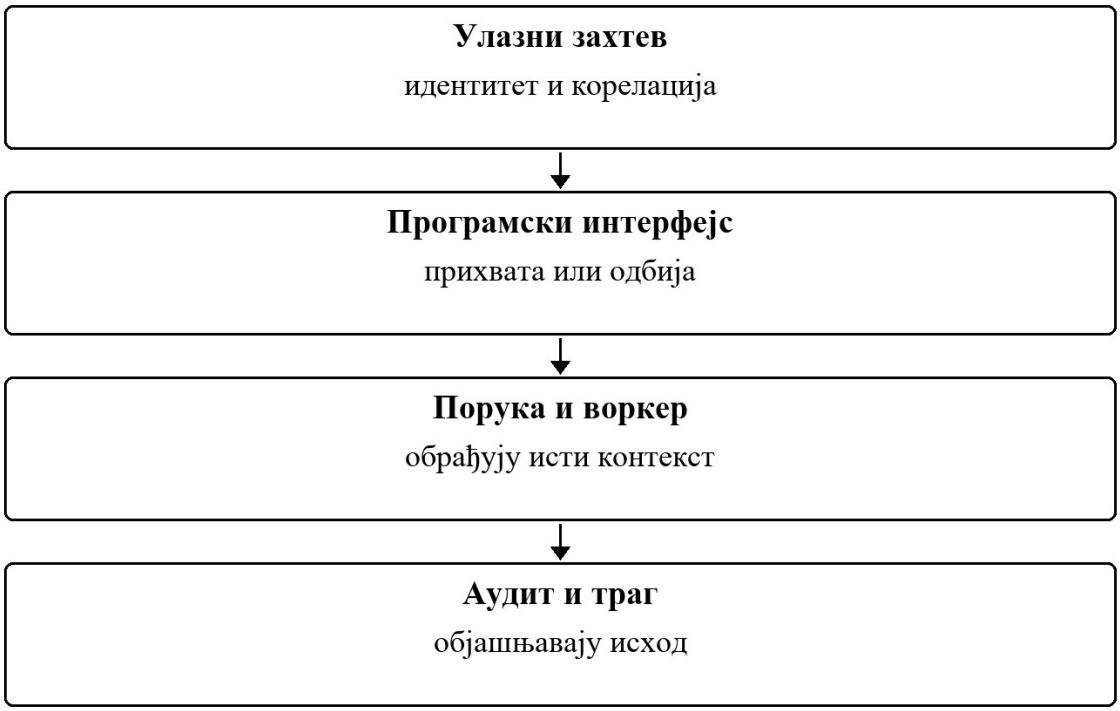
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

3.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

3.5 Шема следљивог дистрибуираног тока



Слика 2. Шема следљивоī дистрибуираноī тока

Један контекст пролази кроз све компоненте, па се неуспех може објаснити без нагађања.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преносе и сценарио отказа који може променити ток.

3.6 Упоредни преглед елемената тока

Табела 1. Елемент и Улоīа у току

Елемент	Улога у току	Питање које треба поставити
Идентитет	ко покреће радњу	Шта се дешава када идентитет није доступан или даје застарелу информацију?
Корелација	један ток кроз компоненте	Шта се дешава када корелација није доступан или даје застарелу информацију?
Аудит	пословно значајна одлука	Шта се дешава када аудит није доступан или даје застарелу информацију?
Конфигурација	услови рада система	Шта се дешава када конфигурација није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

3.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Аудит чува одлуке које су важне за одговорност и истрагу.

2. Логови и трагови чувају технички контекст, али не тајне.

3. Конфигурација има валидно значење, власника и начин промене.

4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Како бисте реконструисали једну неуспелу операцију која је прошла кроз Програмски интерфејс, ред порука и воркер, а да не читате неограничену количину логова?

3.8 Извршиви пример

Пример прати једну операцију кроз две компоненте. Заказивање контакта са станицом добија идентификатор операције (`CorrelationId`) и идентитет актера; исти идентификатор стиже до станице и до ревизијског трага, па се ток може реконструисати и када делови система воде одвојене логове. Ограничење трајања контакта долази из конфигурације која се проверава пре покретања.

Пример 1. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-03-identitet-audit-konfiguracija
dotnet run --project src/Odp.Vezba03.ConsoleUi
dotnet test Odp.Vezba03.sln
```

Правило у коду

Пример 2. examples/odp-dotnet/vezba-03-identitet-audit-konfiguracija/src/Odp.Vezba03.Application/Contacts/ScheduleContactHandler.cs

```
public ScheduleContactResult Schedule(ContactRequest request, OperationContext context)
{
    ArgumentNullException.ThrowIfNull(request);
    ArgumentNullException.ThrowIfNull(context);

    var code = Decide(request, context);

    auditLog.Record(new AuditEntry(
        clock.UtcNow,
        context.CorrelationId,
        context.ActorId,
        Action,
        request.StationId,
        code));

    return new ScheduleContactResult(code == ContactCodes.Scheduled, code,
context.CorrelationId);
}
```

Тест који задаје квар

Пример 3. examples/odp-dotnet/vezba-03-identitet-audit-konfiguracija/tests/Odp.Vezba03.Tests/Application/ScheduleContactHandlerTests.cs

```
[Test]
public void Schedule_CarriesTheSameCorrelationIdToTheStationAndTheAuditTrail()
{
    var result = _handler.Schedule(Request, Context);

    using (Assert.EnterMultipleScope())
    {
        Assert.That(result, Is.EqualTo(new ScheduleContactResult(true,
ContactCodes.Scheduled, "op-42")));
        Assert.That(_stationGateway.Calls.Single().CorrelationId, Is.EqualTo("op-42"));
        Assert.That(
            _auditLog.Entries.Single(),
            Is.EqualTo(new AuditEntry(
                Now,
                "op-42",
                "ana",
                "contact.schedule",
                "GS-NOVI-SAD",
                ContactCodes.Scheduled)));
```

```
    }
}
```

Испис демонстрације

Пример 4. дотнет рун --пројецт src/Odp.Vezba03.ConsoleUi

```
op-0001: GS-NOVI-SAD, 10 min -> contact_scheduled
op-0002: GS-NOVI-SAD, 40 min -> duration_too_long
op-0003: GS-BEOGRAD, 10 min -> station_unavailable

Log stanice:
  op-0001 | GS-NOVI-SAD | reserved=True
  op-0003 | GS-BEOGRAD | reserved=False

Revizijski trag:
  op-0001 | ana | contact.schedule | GS-NOVI-SAD | contact_scheduled
  op-0002 | marko | contact.schedule | GS-NOVI-SAD | duration_too_long
  op-0003 | jelena | contact.schedule | GS-BEOGRAD | station_unavailable
```

Шта треба уочити

- Исти `CorrelationId` види се у логу станице и у ревизијском трагу центра.
- Ревизијски запис настаје и када је захтев одбијен пре позива станице.
- Неисправна конфигурација зауставља покретање са кодом грешке.

Задатак уз пример

Додати у `OperationContext` идентификатор позивајуће компоненте (`Source`) и уписати га у ревизијски запис. Проширити тест тако да доказује да запис садржи и извор операције.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Идентитет, аудит, могућност посматрања система и конфигурација". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.